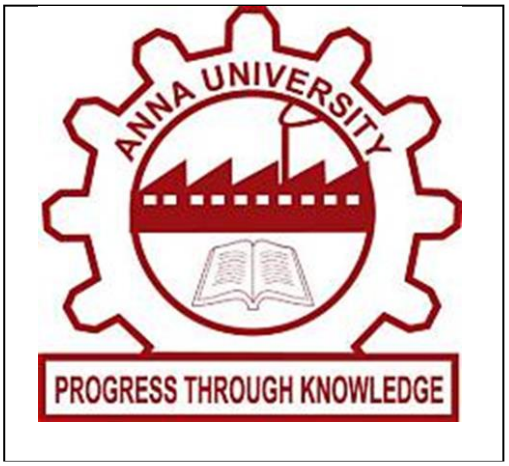
SMART COURSE ENROLLMENT SYSTEM

A SUMMER INTERNSHIP REPORT

Submitted by

NIRAIMATHI M – 810023205046



DEPARTMENT OF INFORMATION TECHNOLOGY

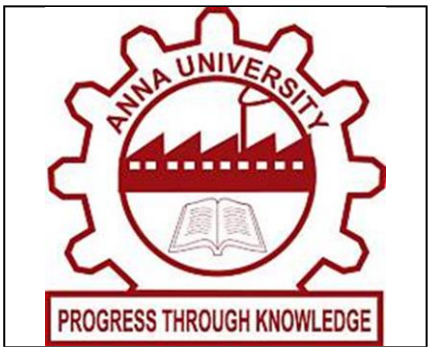
ANNA UNIVERSITY,

TIRUCHIRAPPALLI – 620 024.

UNIVERSITY COLLEGE OF ENGINEERING (BIT CAMPUS),

ANNA UNIVERSITY, TIRUCHIRAPPALLI – 620 024

DEPARTMENT OF INFORMATION TECHNOLOGY



BONAFIDE CERTIFICATE

This is to certify that this Summer Internship project report **"Smart Course Enrollment System"** is the Bonafide work of **NIRAIMATHI M (810023205046)** has satisfactorily completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.TECH INFORMATION TECHNOLOGY** during the academic year **2025-2026**.

Faculty Incharge **Head of the Department**

Submitted for the practical examination held on _____

ABSTRACT

The **Smart Course Enrollment System** is a full-stack web application developed to modernize and simplify the course registration process of training institutions. The system provides a centralized digital platform where learners can browse learning categories, view the course catalog with instructor, duration, level, and seat details, filter courses by category, and submit an enrollment request through an online form. It also provides a login page, a dashboard that summarises key figures, and a panel showing the latest enrollment requests.

The project is implemented using **Python with the FastAPI framework** for the backend, providing a fast and reliable architecture for handling routes, form processing, and database operations. The pages are rendered on the server using the **Jinja2 template engine**, and the frontend is developed using **HTML, CSS, and JavaScript**, providing a responsive and user-friendly interface. **MySQL** is used as the relational database to store categories, courses, and enrollment records, and the application creates its tables and loads sample data automatically on first start.

One of the main objectives of the system is to **reduce manual paperwork, minimize data entry errors, avoid unnecessary waiting time, and make course information easily available to learners**. By digitizing processes such as course discovery, filtering, and enrollment, the system improves operational efficiency and provides a convenient experience for both learners and staff.

Overall, the **Smart Course Enrollment System** demonstrates the practical application of full-stack development and modern software engineering principles. It combines a Python backend with an interactive web interface to provide an effective solution for managing course enrollment. The project highlights how technology can be used to automate enrollment operations, improve record keeping, and enhance the overall efficiency of course management.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
1	INTRODUCTION 1.1 OBJECTIVES OF THE INTERNSHIP 1.2 MOTIVATION	1
2	COMPANY PROFILE About the Company: Company Details: Services Offered Technical Domains Technologies Used by the Company	4
3	DAILY WORK REPORT	6
4	BLOCK DIAGRAM	9
5	SMART COURSE ENROLLMENT SYSTEM 5.1 Existing System 5.2 Proposed System 5.3 Capabilities	10
6	PROPOSED METHOD 6.1 Home and Category Management 6.2 Course Catalog Management 6.3 Enrollment Management 6.4 Login Management 6.5 Dashboard Management 6.6 Contact Management 6.7 Database Management 6.8 Notification and Status Management	12
7	ALGORITHM 7.1 Browse Categories – Algorithm 7.2 Course Catalog and Filter – Algorithm 7.3 Enrollment – Algorithm 7.4 User Login – Algorithm 7.5 Database Initialisation – Algorithm 7.6 Dashboard – Algorithm	14
8	CODE IMPLEMENTATION 8.1 Application Setup and Database Connection Code 8.2 Database Initialisation Code 8.3 Data Model and Helper Functions Code 8.4 Home and Categories Module Code 8.5 Course Catalog Module Code 8.6 Enrollment Module Code 8.7 Login Module Code 8.8 Dashboard and Contact Module Code 8.9 JavaScript Code 8.10 Stylesheet Code (Colour Theme)	16

9	SAMPLE OUTPUT 9.1 Home Page 9.2 Login Page 9.3 Categories Page 9.4 Course Catalog Page 9.5 Enrollment Page 9.6 Successful Enrollment	27
10	ANALYSIS 10.1 Observations	30
11	CONCLUSION	32
12	FUTURE WORK	33
13	REFERENCES	34

1. INTRODUCTION

The **Smart Course Enrollment System** is a Python full stack web application developed to automate and simplify the course registration process of training institutions and learning platforms. In today's digital environment, learners expect to explore courses, understand what each course offers, and register for it within a few minutes using a phone or a laptop. Traditional enrollment methods that depend on paper forms, spreadsheets, phone calls, and manual record keeping often face problems such as long waiting times, scattered course information, data entry errors, duplicate records, and difficulty in tracking how many learners have applied for each course. To overcome these challenges, a centralized and web-based **Smart Course Enrollment System** has been developed.

The primary purpose of this project is to provide a simple and user-friendly platform where learners can browse learning categories, view the course catalog with complete details, filter courses by category, and submit an enrollment request online. The system also provides a login page and a dashboard that summarises the number of categories, courses, and recent enrollments, along with the list of available courses and the latest enrollment requests. This automation reduces manual work, improves data accuracy, and enhances the overall experience of learners.

The application is developed using **Python full stack technologies**. The frontend is designed using **HTML5, CSS3, and JavaScript**, which provide an attractive, responsive, and interactive user interface with a dark gradient theme and card-based layout. The backend is developed using the **FastAPI** framework, which handles routing, business logic, form processing, and communication between the pages and the database. Pages are rendered on the server using the **Jinja2** template engine. **MySQL** is used as the relational database management system to store categories, courses, and enrollment records, and the **mysql-connector-python** driver connects the application to the database. The application is served using **Uvicorn**, and **Visual Studio Code** was used as the development environment.

The system follows a **three-tier architecture**, consisting of the Presentation Layer (HTML templates, CSS, and JavaScript in the browser), the Business Logic Layer (FastAPI backend), and the Database Layer (MySQL). The browser sends requests to the backend, the backend reads from or writes to the database, and the result is returned to the user as a

rendered web page. This layered architecture improves the maintainability, scalability, and reliability of the system.

The **Smart Course Enrollment System** consists of several important modules, including **Home and Category Management, Course Catalog and Filter, Online Enrollment, Login, Dashboard, Contact, and Database Management**. Each module performs specific functions and works together to provide a complete course enrollment solution.

Overall, the **Smart Course Enrollment System** is a practical implementation of Python full stack development that demonstrates the integration of frontend technologies, FastAPI backend services, and MySQL database management. The project provides practical knowledge of web application development, database connectivity, form handling, template rendering, and software engineering principles. It serves as an efficient and user-friendly solution for training institutions and can be further enhanced with features such as **secure authentication, an administrator panel, seat management, email and SMS notifications, and online payment** in future versions.

1.1 OBJECTIVES OF THE INTERNSHIP

Specific Objectives

The main objective of the **Smart Course Enrollment System** is to develop a secure, user-friendly, and efficient web application using **Python full stack technologies**. The system is designed to simplify the course enrollment process by providing an online platform where learners can explore courses and submit their enrollment requests effectively.

The objectives of the project are:

- To develop a web-based application using **Python, FastAPI, Jinja2, HTML, CSS, JavaScript, and MySQL**.
- To design a **relational database** that stores categories, courses, and enrollment records.
- To enable learners to **view learning categories and courses** with instructor, duration, level, and seat details.
- To allow learners to **filter the course catalog by category**.
- To provide an **online enrollment form** with input validation that stores requests in the database.

- To provide a **login screen** with basic input validation as a base for future authentication.
- To show a **dashboard** with summary figures and recent enrollments.
- To **reduce manual paperwork** and minimize data entry errors.
- To provide a **responsive, reliable, and easy-to-use** web interface for desktop and mobile screens.
- To **automate database setup** so that the application can be installed and run with minimum effort.
- To demonstrate the practical application of **Python full stack development, database connectivity, and software engineering principles**.

1.2 MOTIVATION

The motivation behind developing the **Smart Course Enrollment System** is to make the course registration process faster, easier, and more efficient. Many institutions still depend on manual methods to publish course information and collect learner details, which can lead to delays, inaccurate records, duplicate entries, and unnecessary effort for both learners and staff.

This project provides an online platform where learners can easily **explore categories, compare courses, and enroll with a few clicks**. It also stores all enrollment requests in a single centralized database, so that administrators can view and process them from one place.

Another major motivation is to **reduce paperwork, minimize manual errors, and provide better convenience to learners**. By automating the important steps of enrollment, the system saves time and improves the overall efficiency of course management.

This project also provides practical knowledge of **Python full stack development** by using technologies such as **FastAPI, Jinja2, HTML, CSS, JavaScript, and MySQL** to develop a real-world web application. It helps demonstrate how modern software technologies can be applied to solve common problems in the education and training domain.

2. COMPANY PROFILE



About the Company:

Astonish InfoTech Private Limited is a software development and web solutions company located in Tiruchirappalli (Trichy), Tamil Nadu, India. The company delivers creative, reliable, and secure digital solutions for startups, institutions, and businesses. Its development team builds responsive websites, software applications, ERP systems, CRM platforms, mobile apps, automation tools, and cloud-ready solutions that help organizations improve their operational efficiency and adopt digital technology.

The company follows a simple four-step development process: **Discuss** (requirement analysis), **Design** (UI/UX and planning), **Develop** (coding and testing), and **Deploy** (launch and support). Its work covers custom software development, secure database solutions, professional UI/UX design, and maintenance and support after delivery.

Astonish InfoTech also works in advanced technical domains such as Artificial Intelligence and Machine Learning, Embedded Systems, IoT solutions, VLSI support, Cloud solutions, and Cyber Security. The company offers ready-to-customize products such as School ERP, Billing Software, Hospital Management, E-Commerce System, CRM Software, and Inventory System. It also handles project areas such as education software, retail business software, industrial automation, office management, digital marketing, and database projects.

The company has a team of developers, designers, and analysts working in roles such as Fullstack Developer, Backend Developer, Frontend Developer, AI/ML Developer, Data Analyst, and UI/UX Developer, led by its Founder and CEO.

As part of my internship at Astonish InfoTech Private Limited, I developed the Smart Course Enrollment System using Python full stack technologies. The project involved designing a responsive and user-friendly interface using HTML, CSS, and JavaScript, developing backend services using Python and FastAPI, integrating the MySQL database, rendering pages with Jinja2 templates, and testing the application. The system was designed to simplify course enrollment by providing features such as category browsing, a course catalog with filtering, online enrollment, login, and a dashboard.

The internship provided me with valuable practical knowledge of Python full stack development and helped me understand the complete Software Development Life Cycle

(SDLC). It improved my programming skills, database handling, problem-solving abilities, and understanding of frontend-backend integration.

Overall, the internship at Astonish InfoTech Private Limited provided an excellent opportunity to apply theoretical concepts in a practical software development environment. The experience helped me understand industry-oriented development practices and strengthened my technical skills in Python, FastAPI, MySQL, HTML, CSS, and JavaScript.

Company Details:

- **Company Name:** Astonish InfoTech Private Limited
- **Industry:** Information Technology (IT) Services, Software and Web Development
- **Location:** G-7, Technology Park (BUTP), Bharathidasan University, Kajamalai Campus, near Law College, Tiruchirappalli (Trichy) – 620 023, Tamil Nadu, India
- **Development Technologies:** HTML, CSS, JavaScript, React, PHP, Python, MySQL, Node.js, Bootstrap, APIs, Cloud, and IoT
- **Major Services:** Web Development, Software Development, Mobile App Development, ERP Solutions, CRM Solutions, and Domain and Hosting
- **Internship Duration:** 01/06/2026 to 30/06/2026
- **Development Approach:** Creative, reliable, and secure software solutions built through the Discuss, Design, Develop, and Deploy process
- **Project Completed During Internship:** Smart Course Enrollment System (Python Full Stack)

Services Offered

- Web Development – responsive business websites, landing pages, and dynamic web portals
- Software Development – custom software for billing, inventory, accounts, and business automation
- Mobile App Development – Android and mobile-friendly applications
- ERP Solutions – school ERP, college ERP, office management, and enterprise software
- CRM Solutions – lead management, customer tracking, follow-up, and sales dashboards

- Domain and Hosting – domain registration, hosting, SSL, business email, and deployment

Technical Domains

- AI and Machine Learning
- Embedded Systems
- IoT Solutions
- VLSI Support
- Cloud Solutions
- Cyber Security

Technologies Used by the Company

- **Frontend:** HTML, CSS, JavaScript, React, Bootstrap
- **Backend:** PHP, Python, Node.js
- **Database:** MySQL
- **Other:** APIs, Cloud, and IoT

3. DAILY WORK REPORT

Day 1: Attended the orientation session and was briefed on the internship guidelines, evaluation criteria, and reporting format. Discussed and finalized the project topic, “Smart Course Enrollment System”, with the mentor.

Day 2: Conducted requirement gathering by discussing with the mentor. Listed the stakeholders (Learner and Administrator) and identified the core functional requirements of the course enrollment application.

Day 3: Carried out a study of existing course registration methods and online learning portals. Studied their features and documented the drawbacks of manual, paper-based enrollment.

Day 4: Finalized the scope and objectives of the project. Prepared the list of hardware and software requirements needed for development.

Day 5: Set up the development environment: installed Python, Visual Studio Code, MySQL Server, and MySQL Workbench. Created a virtual environment for the project.

Day 6: Studied FastAPI fundamentals: creating an application, defining routes with decorators, handling GET and POST requests, and running the server with Uvicorn.

Day 7: Studied the Jinja2 template engine: variables, loops, conditions, and rendering HTML templates from FastAPI route functions.

Day 8: Designed the overall system architecture using a layered approach (Presentation Layer, Business Logic Layer, and Database Layer) and prepared the block diagram.

Day 9: Designed the database schema by identifying the entities Category, Course, and Enrollment, along with their attributes and relationships.

Day 10: Prepared the Entity-Relationship (ER) diagram showing the one-to-many relationship between categories and courses, and wrote the SQL script (database.sql) to create the tables.

Day 11: Created wireframes for the Home, Categories, Courses, Enrollment, and Login pages. Planned the responsive layout and the colour theme.

Day 12: Wrote the common stylesheet (style.css) with CSS variables, gradient background, top navigation bar, cards, buttons, and form styles.

Day 13: Developed the Home page template (index.html) with the navigation bar, hero section, statistics cards, and popular categories.

Day 14: Initialized the FastAPI project structure with the app.py file, static folder, and templates folder. Mounted the static files and configured the template engine.

Day 15: Implemented the database connection function and the fetch_all helper using mysql-connector-python with dictionary cursors.

Day 16: Implemented the init_db function to create the tables automatically and to load sample categories and courses on the first run.

Day 17: Added migration checks in init_db to add missing columns to an older enrollments table, and added error handling for database connection failures.

Day 18: Developed the Categories module: the route, the SQL query, and the categories.html template with category cards and links.

Day 19: Developed the Course Catalog module with a JOIN query between courses and categories, and displayed the course details in courses.html.

Day 20: Implemented the category filter on the course catalog using a query parameter and a parameterised WHERE clause.

Day 21: Developed the Enrollment page (enroll.html) with fields for name, email, phone, category, and course, and added HTML validation attributes.

Day 22: Implemented the POST route for enrollment that reads the form data, inserts the record into the enrollments table, and shows a success or error message.

Day 23: Developed the Login page and the login route with validation for empty fields and minimum password length, followed by redirection to the dashboard.

Day 24: Developed the Dashboard page showing summary statistics, the list of available courses, and the latest enrollment requests.

Day 25: Developed the Contact page and added the footer script (script.js) that displays the current year automatically.

Day 26: Performed functional testing of all pages and forms in the browser, and checked the stored records in MySQL Workbench.

Day 27: Tested validation scenarios such as empty fields, invalid email format, and short passwords. Fixed layout issues on smaller screens.

Day 28: Reviewed the code, identified limitations such as duplicate sample data and missing authentication, and documented the improvements for future work.

Day 29: Captured screenshots of all the working pages and prepared the output section of the report.

Day 30: Completed the documentation, compiled the final internship report, and prepared for the final presentation.

4. BLOCK DIAGRAM

The block diagram shows how the learner, the backend system, and the database interact with each other. The learner opens the web pages, views categories and courses, and submits the enrollment form. Each request is received by a route handler in the FastAPI application, which validates the input, runs the required SQL query on the MySQL database, renders a Jinja2 template, and sends the HTML page back to the browser.

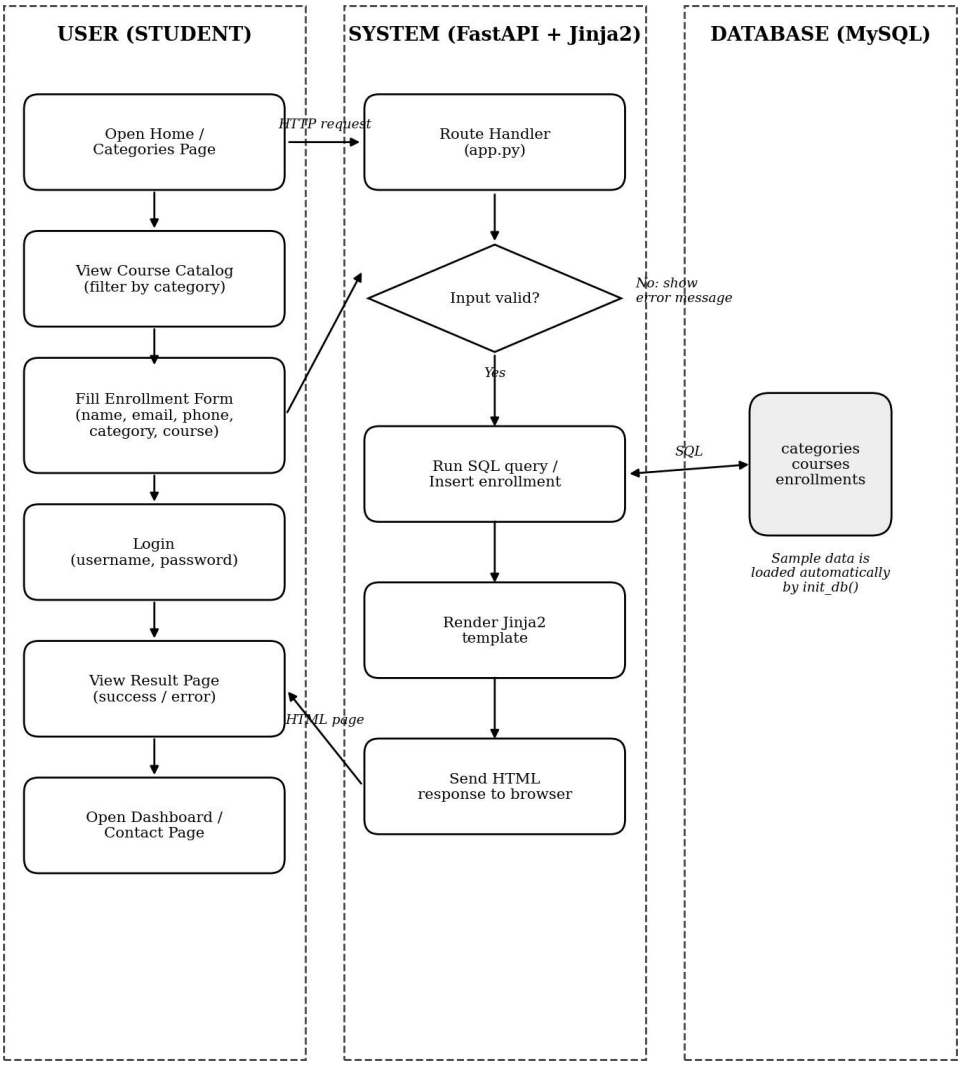


Figure 4.1: Block diagram of the Smart Course Enrollment System

5. SMART COURSE ENROLLMENT SYSTEM

5.1 Existing System

Most traditional training centres and institutions still operate through manual and semi-manual processes for publishing course information and registering learners. Learners typically need to visit the institution directly or depend on staff to know which courses are available. Course details, learner information, and enrollment records are often maintained using paper forms, registers, or separate spreadsheets, with limited automation and centralized management.

This existing approach suffers from several limitations:

- **Manual enrollment process** – Learners fill paper forms, and staff re-enter the details into registers or spreadsheets, increasing the chances of spelling mistakes, duplicate entries, and missing records.
- **Scattered course information** – Details such as instructor, duration, level, and available seats are spread across brochures and notice boards, which quickly become outdated.
- **Limited accessibility** – Learners can enroll only during office hours and usually only in person or by phone.
- **Fragmented record keeping** – Learner details and enrollment information are maintained in separate files, making data retrieval difficult and time-consuming.
- **No instant confirmation** – Learners do not receive immediate confirmation that their request has been recorded.
- **Difficulty in tracking enrollments** – It is hard to find out how many learners have applied for a particular course or category.
- **Inefficient administrative management** – Preparing summaries and reports requires manual counting.
- **Increased waiting time** – Queues form at the counter during admission periods, reducing the efficiency of the institution.

These drawbacks make the existing manual system less efficient for handling a growing number of learners and courses. Therefore, there is a need for a digital, centralized, and user-friendly Smart Course Enrollment System that can automate the enrollment process, reduce manual errors, and provide convenient service to learners.

5.2 Proposed System

The proposed **Smart Course Enrollment System** is a full stack web application that digitizes and centralizes the major operations of course enrollment. It replaces traditional manual procedures with an integrated online platform that can be accessed by learners and administrators from any device with a browser. The system is designed to simplify course discovery, reduce waiting time, minimize data entry errors, and improve the overall efficiency of the enrollment process.

The system is developed using **Python and FastAPI** for the backend and **HTML, CSS, and JavaScript** for the frontend, with **MySQL** used for centralized data storage. Key aspects of the proposed system include:

- **Centralized platform** – Categories, courses, and enrollment records are stored and managed in a single centralized database.
- **Category browsing** – Learners can view all learning categories with an icon and description, and open a category to see its courses.
- **Course catalog with filtering** – Learners can view each course with its description, instructor, duration, level, and seats, and filter the catalog by category.
- **Online enrollment** – Learners can submit their name, email, phone number, category, and preferred course through a simple online form.
- **Instant feedback** – The system shows a success message after a successful enrollment and an error message if something goes wrong.
- **Login page** – A login screen validates the username and password fields and takes the user to the dashboard.
- **Dashboard** – Staff can see summary figures, the available courses, and the latest enrollment requests on a single page.
- **Automatic database setup** – The application creates the required tables and loads sample categories and courses on first start.
- **Responsive design** – The layout adapts to different screen widths, so the application can be used on desktop and mobile devices.

By automating these processes, the proposed Smart Course Enrollment System reduces manual workload, minimizes data entry errors, and provides a convenient and efficient enrollment experience for both learners and staff.

5.3 Capabilities

The Smart Course Enrollment System offers the following key capabilities:

- **Category Management** – Displays all learning categories with icons, colours, and descriptions loaded from the database.
- **Course Catalog** – Lists courses with title, description, category, instructor, duration, level, and number of seats.
- **Course Filtering** – Allows learners to view only the courses that belong to a selected category.
- **Online Enrollment** – Enables learners to submit enrollment requests, which are stored in the enrollments table with a default status of Pending.

- **Login and Navigation** – Provides a login page with input validation and a consistent navigation bar on all pages.
- **Dashboard and Monitoring** – Helps staff see the number of categories, courses, and recent enrollments, and review the latest requests.
- **Contact Information** – Shows the support email, phone number, and working hours.
- **Database Management** – Stores categories, courses, and enrollments securely in a centralized MySQL database using parameterised queries.
- **Automatic Initialisation** – Creates tables, upgrades older tables, and inserts sample data automatically when the application starts.

6. PROPOSED METHOD

6.1 Home and Category Management

- Displays the home page with the platform introduction, summary statistics, and popular categories.
- Reads all categories from the database and shows each one as a card with an icon, name, and description.
- Each category card contains an “Open category” link that opens the course catalog already filtered for that category.
- Uses the colour stored for each category to give its icon a matching soft background.

6.2 Course Catalog Management

- Joins the courses table with the categories table to show the category name along with each course.
- Shows course details such as title, description, instructor, duration, level, and available seats.
- Provides a dropdown filter so that learners can choose a category and view only its courses.
- Passes the selected category to the database query as a parameter, which protects against SQL injection.

6.3 Enrollment Management

- Displays an enrollment form with fields for full name, email address, phone number, category, and preferred course.
- Uses HTML validation so that name, email, category, and course are required and the email has a valid format.
- Reads the submitted values on the server, removes extra spaces, and inserts a new record into the enrollments table.
- Sets the status of every new enrollment to Pending and records the time of submission automatically.
- Displays a success message after saving, or an error message if the operation fails.

6.4 Login Management

- Provides a login page with fields for email or username and password.
- Checks that both fields are filled and that the password has at least four characters.
- Displays an appropriate error message if the validation fails.
- Redirects the user to the dashboard after valid input.
- Includes a link for new users to open the enrollment page.

6.5 Dashboard Management

- Shows summary cards for the number of categories, courses, and recent enrollments.
- Lists all available courses with a link to the full course catalog.
- Lists the latest enrollment requests with the student name, course, category, and status.

6.6 Contact Management

- Displays the support email, phone number, and working hours.
- Helps learners to reach the support team for questions about courses and enrollment.

6.7 Database Management

- Stores categories, courses, and enrollments in a centralized MySQL database.

- Maintains a one-to-many relationship between categories and courses using a foreign key.
- Creates the tables automatically and adds missing columns to older tables.
- Provides efficient data retrieval and updating through the FastAPI backend.
- Helps maintain data consistency, accuracy, and security throughout the application.

6.8 Notification and Status Management

- Displays a success message after a successful enrollment.
- Displays clear error messages for failed enrollments and invalid login input.
- Records the status of each enrollment, which is Pending by default, so that it can be reviewed later.
- Prints detailed error information in the server console to help the developer find problems quickly.

7. ALGORITHM

7.1 Browse Categories – Algorithm

1. User opens the home page or the categories page.
2. The browser sends a GET request to the server.
3. The route function runs a query to read all categories.
4. The results are passed to the Jinja2 template.
5. The template creates one card for each category.
6. The rendered page is returned to the browser.
7. The user clicks Open category to view the courses of that category.

7.2 Course Catalog and Filter – Algorithm

1. User opens the course catalog page, with or without a category selected.
2. The server reads the category value from the query string.
3. The server prepares a query that joins the courses and categories tables.
4. If a category is selected, a WHERE clause is added and the value is passed as a parameter.
5. The query is executed and the matching courses are fetched.

6. The list of all categories is also fetched for the filter dropdown.
7. The template displays the filter form and the list of matching courses.

7.3 Enrollment – Algorithm

1. User opens the enrollment page, and the system loads categories and courses into the dropdown lists.
2. User enters name, email, phone number, category, and course.
3. The browser checks the required fields and the email format.
4. The form is submitted to the server using the POST method.
5. The server reads the form values and removes extra spaces.
6. The server opens a database connection and inserts a new row into the enrollments table.
7. If the insert succeeds, the page is shown again with a success message.
8. If an error occurs, the error is printed in the console and an error message is shown to the user.

7.4 User Login – Algorithm

1. User enters email or username and password in the login form.
2. The form is submitted to the server using the POST method.
3. The server reads the username and password and removes extra spaces.
4. If either field is empty, an error message is displayed.
5. If the password has fewer than four characters, an error message is displayed.
6. If both checks pass, the user is redirected to the dashboard using an HTTP 303 redirect.

7.5 Database Initialisation – Algorithm

1. The application starts and calls the initialisation function.
2. The function opens a connection to the MySQL database.
3. The tables categories, courses, and enrollments are created if they do not exist.
4. The function checks whether the columns category_name, phone, and status exist in the enrollments table, and adds any that are missing.
5. If the categories table is empty, the sample categories are inserted.

6. If the courses table is empty, the sample courses are inserted.
7. The connection is closed. If a database error occurs, a message is printed and the application continues to start.

7.6 Dashboard – Algorithm

1. User opens the dashboard page.
2. The server counts the categories, courses, and recent enrollments.
3. The server reads all categories, all courses, and the latest eight enrollments.
4. The data is passed to the dashboard template.
5. The template shows the summary cards, the course list, and the recent enrollment list.

8. CODE IMPLEMENTATION

8.1 Application Setup and Database Connection Code

Creates the FastAPI application, mounts the static files, registers the template folder, and provides the function that opens a connection to the MySQL database.

```
import traceback
from fastapi import FastAPI, Request
from fastapi.responses import HTMLResponse, RedirectResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from pydantic import BaseModel
import mysql.connector

app = FastAPI(title="Smart Course Enrollment Platform", version="2.0.0")
app.mount("/static", StaticFiles(directory="static"), name="static")
templates = Jinja2Templates(directory="templates")

def get_db_connection():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="********",
        database="course_enrollment",
        autocommit=True,
    )
```

8.2 Database Initialisation Code

Creates the tables, upgrades older tables, and inserts the sample categories and courses when the application starts.

```
def init_db():
    try:
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS categories (
                id INT AUTO_INCREMENT PRIMARY KEY,
                name VARCHAR(100) NOT NULL UNIQUE,
                description TEXT,
                image VARCHAR(100),
                color VARCHAR(30),
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )
        """)
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS courses (
                id INT AUTO_INCREMENT PRIMARY KEY,
                title VARCHAR(150) NOT NULL,
                category_id INT NOT NULL,
                instructor VARCHAR(100),
                duration VARCHAR(50),
                level VARCHAR(50),
                description TEXT,
                seats INT DEFAULT 50,
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                FOREIGN KEY (category_id) REFERENCES categories(id)
            )
        """)
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS enrollments (
                id INT AUTO_INCREMENT PRIMARY KEY,
                student_name VARCHAR(100) NOT NULL,
                student_email VARCHAR(100) NOT NULL,
                course_name VARCHAR(150) NOT NULL,
                category_name VARCHAR(100),
                phone VARCHAR(30),
                status VARCHAR(30) DEFAULT 'Pending',
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )
        """)

        # Migrate old enrollments table if columns are missing
        cursor.execute("SHOW COLUMNS FROM enrollments LIKE
'category_name'")
        if cursor.fetchone() is None:
            cursor.execute("ALTER TABLE enrollments ADD COLUMN
category_name VARCHAR(100) NULL AFTER course_name")
        cursor.execute("SHOW COLUMNS FROM enrollments LIKE 'phone'")
        if cursor.fetchone() is None:
            cursor.execute("ALTER TABLE enrollments ADD COLUMN phone
VARCHAR(30) NULL AFTER category_name")
        cursor.execute("SHOW COLUMNS FROM enrollments LIKE 'status'")
        if cursor.fetchone() is None:
            cursor.execute("ALTER TABLE enrollments ADD COLUMN status
```

```
VARCHAR(30) DEFAULT 'Pending' AFTER phone")

        cursor.execute("SELECT COUNT(*) FROM categories")
        if cursor.fetchone()[0] == 0:
            cursor.execute("""
                INSERT INTO categories (name, description, image, color)
VALUES
                ('Computer Science', 'Modern programming, engineering,
and AI-focused learning paths.', '[icon]', '#4f46e5'),
                ('Business & Leadership', 'Strategy, management, and
entrepreneurial growth for ambitious teams.', '[icon]', '#0f766e'),
                ('Design & Creative', 'Visual storytelling, product
design, and creative thinking workshops.', '[icon]', '#db2777'),
                ('Data Science', 'Analytics, machine learning, and
decision-making for data-driven careers.', '[icon]', '#ea580c'),
                ('Languages & Communication', 'Professional
communication and language confidence for global careers.', '[icon]',
'#7c3aed')
            """)

        cursor.execute("SELECT COUNT(*) FROM courses")
        if cursor.fetchone()[0] == 0:
            cursor.execute("""
                INSERT INTO courses (title, category_id, instructor,
duration, level, description, seats) VALUES
                ('Full-Stack Web Development', 1, 'Asha Khan', '8
Weeks', 'Beginner', 'Build modern web apps with HTML, CSS, JavaScript,
and Python.', 30),
                ('AI & Machine Learning Essentials', 1, 'Ravi Sharma',
'10 Weeks', 'Intermediate', 'Learn practical machine learning workflows
and real-world projects.', 25),
                ('Leadership for Growth Teams', 2, 'Mina Patel', '6
Weeks', 'Intermediate', 'Master leadership habits that scale teams and
ideas.', 20),
                ('UI/UX Design System Mastery', 3, 'Noor Ali', '7
Weeks', 'Beginner', 'Create polished interfaces that feel intuitive and
premium.', 22),
                ('Data Visualization with Power BI', 4, 'Daniel Brooks',
'5 Weeks', 'Beginner', 'Turn raw data into meaningful insights and
stunning dashboards.', 18),
                ('Business English Communication', 5, 'Sofia Martin', '4
Weeks', 'Beginner', 'Improve confidence in presentations, networking,
and interviews.', 24)
            """)

        cursor.close()
        conn.close()
    except mysql.connector.Error as exc:
        print(f"Database initialization skipped: {exc}")


init_db()
```

8.3 Data Model and Helper Functions Code

Defines the enrollment data model and the helper functions used to read data from the database and to calculate the statistics shown on the pages.

```
class EnrollmentPayload(BaseModel):
    student_name: str
    student_email: str
    course_name: str
    category_name: str
    phone: str | None = None


def fetch_all(query: str, params=()):
    conn = get_db_connection()
    cursor = conn.cursor(dictionary=True)
    cursor.execute(query, params)
    result = cursor.fetchall()
    cursor.close()
    conn.close()
    return result


def get_stats():
    categories = fetch_all("SELECT * FROM categories")
    courses = fetch_all("SELECT * FROM courses")
    enrollments = fetch_all("SELECT * FROM enrollments ORDER BY id DESC
LIMIT 8")
    return {
        "categories": len(categories),
        "courses": len(courses),
        "enrollments": len(enrollments),
    }
```

8.4 Home and Categories Module Code

The home route reads all categories and the summary statistics. The categories route reads all categories for the categories page.

```
@app.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse(request, "index.html", {
        "request": request,
        "categories": fetch_all("SELECT * FROM categories ORDER BY id"),
        "stats": get_stats(),
    })

@app.get("/categories", response_class=HTMLResponse)
async def categories_page(request: Request):
    return templates.TemplateResponse(request, "categories.html", {
        "request": request,
```

```
        "categories": fetch_all("SELECT * FROM categories ORDER BY id"),
    })
```

categories.html (category cards)

```
{% for category in categories %}
<article class="card">
  <div class="card-icon" style="background: {{ category.color }}15;
       color: {{ category.color }};">{{ category.image }}</div>
  <h3>{{ category.name }}</h3>
  <p>{{ category.description }}</p>
  <a href="/courses?category={{ category.name }}">Open category</a>
</article>
{% endfor %}
```

8.5 Course Catalog Module Code

Reads the optional category filter, joins the courses and categories tables, and passes the result to the template.

```
@app.get("/courses", response_class=HTMLResponse)
async def courses_page(request: Request):
    selected_category = request.query_params.get("category")
    query = """
        SELECT c.*, cat.name AS category_name
        FROM courses c
        JOIN categories cat ON c.category_id = cat.id
    """
    params = ()
    if selected_category:
        query += " WHERE cat.name = %s"
        params = (selected_category,)
    query += " ORDER BY c.id"
    return templates.TemplateResponse(request, "courses.html", {
        "request": request,
        "courses": fetch_all(query, params),
        "categories": fetch_all("SELECT * FROM categories ORDER BY id"),
        "selected_category": selected_category,
    })
```

courses.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"
/>
  <title>Courses | EduSphere</title>
  <link rel="stylesheet" href="/static/style.css" />
</head>
<body>
  <header class="topbar">
```

```
    <a class="brand" href="/">EduSphere</a>
    <nav class="nav-links">
      <a href="/">Home</a>
      <a href="/categories">Categories</a>
      <a href="/courses">Courses</a>
      <a href="/enroll">Enroll</a>
      <a href="/login">Login</a>
    </nav>
  </header>

  <main>
    <section class="section-header">
      <h2>Our Course Catalog</h2>
      <a href="/enroll">Reserve a seat</a>
    </section>

    <form class="panel" method="get" action="/courses" style="margin-
bottom: 16px;">
      <label for="category">Filter by category</label>
      <select id="category" name="category">
        <option value="">All categories</option>
        {% for category in categories %}
        <option value="{{ category.name }}" {% if selected_category ==
category.name %}selected{% endif %}>{{ category.name }}</option>
        {% endfor %}
      </select>
      <button type="submit" style="margin-top: 10px;">Apply</button>
    </form>

    <section class="course-list">
      {% for course in courses %}
      <article class="course-item">
        <h3>{{ course.title }}</h3>
        <p class="muted">{{ course.description }}</p>
        <div class="muted">Category: {{ course.category_name }} •
Instructor: {{ course.instructor }} • Duration: {{ course.duration }} •
Level: {{ course.level }} • Seats: {{ course.seats }}</div>
      </article>
      {% endfor %}
    </section>
  </main>

  <p class="footer-note">© <span data-year></span> EduSphere. All rights
reserved.</p>
  <script src="/static/script.js"></script>
</body>
</html>
```

8.6 Enrollment Module Code

The GET route shows the enrollment form. The POST route reads the submitted data, inserts a new enrollment, and shows a success or error message.

```
@app.get("/enroll", response_class=HTMLResponse)
async def enroll_page(request: Request):
    return templates.TemplateResponse(request, "enroll.html", {
        "request": request,
        "categories": fetch_all("SELECT * FROM categories ORDER BY id"),
        "courses": fetch_all("SELECT * FROM courses ORDER BY id"),
    })


@app.post("/enroll", response_class=HTMLResponse)
async def enroll_student(request: Request):
    try:
        form_data = await request.form()
        student_name = str(form_data.get("student_name", "")).strip()
        student_email = str(form_data.get("student_email", "")).strip()
        course_name = str(form_data.get("course_name", "")).strip()
        category_name = str(form_data.get("category_name", "")).strip()
        phone = str(form_data.get("phone", "") or "")

        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute("""
            INSERT INTO enrollments (student_name, student_email,
course_name, category_name, phone)
            VALUES (%s, %s, %s, %s, %s)
        """, (student_name, student_email, course_name, category_name,
phone))
        conn.commit()
        cursor.close()
        conn.close()
        return templates.TemplateResponse(request, "enroll.html", {
            "request": request,
            "categories": fetch_all("SELECT * FROM categories ORDER BY
id"),
            "courses": fetch_all("SELECT * FROM courses ORDER BY id"),
            "success": "Enrollment submitted successfully. We will
contact you shortly.",
        })
    except Exception as exc:
        traceback.print_exc()
        error_text = str(exc) or "Unexpected server error."
        return templates.TemplateResponse(request, "enroll.html", {
            "request": request,
            "categories": fetch_all("SELECT * FROM categories ORDER BY
id"),
            "courses": fetch_all("SELECT * FROM courses ORDER BY id"),
            "error": error_text,
        })
```

enroll.html

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"
/>
  <title>Enroll | EduSphere</title>
  <link rel="stylesheet" href="/static/style.css" />
</head>
<body>
  <header class="topbar">
    <a class="brand" href="/">EduSphere</a>
    <nav class="nav-links">
      <a href="/">Home</a>
      <a href="/categories">Categories</a>
      <a href="/courses">Courses</a>
      <a href="/enroll">Enroll</a>
      <a href="/login">Login</a>
    </nav>
  </header>

  <main>
    <section class="hero">
      <div class="hero-content">
        <p class="eyebrow">Enrollment Portal</p>
        <h1>Reserve your seat in a next-level learning experience.</h1>
        <p>Choose a category, select an exciting program, and share your
details for a guided onboarding experience.</p>
      </div>
    </section>

    {% if success %}
    <div class="success">{{ success }}</div>
    {% endif %}

    <section class="panel" style="margin-top: 20px;">
      <form method="post" action="/enroll">
        <label for="student_name">Full Name</label>
        <input id="student_name" name="student_name" type="text"
required />

        <label for="student_email">Email Address</label>
        <input id="student_email" name="student_email" type="email"
required />

        <label for="phone">Phone Number</label>
        <input id="phone" name="phone" type="tel" />

        <label for="category_name">Category</label>
        <select id="category_name" name="category_name" required>
          {% for category in categories %}
          <option value="{{ category.name }}">{{ category.name
}}</option>
          {% endfor %}
        </select>
```

```
        <label for="course_name">Preferred Course</label>
        <select id="course_name" name="course_name" required>
          {% for course in courses %}
          <option value="{{ course.title }}">{{ course.title }}</option>
          {% endfor %}
        </select>

        <button type="submit">Submit Enrollment</button>
      </form>
    </section>
  </main>

  <p class="footer-note">© <span data-year></span> EduSphere. All rights
reserved.</p>
  <script src="/static/script.js"></script>
</body>
</html>
```

8.7 Login Module Code

Validates the login input and redirects the user to the dashboard.

```
@app.get("/login", response_class=HTMLResponse)
async def login_page(request: Request):
    return templates.TemplateResponse(request, "login.html", {"request":
request, "error": None})


@app.post("/login", response_class=HTMLResponse)
async def login_submit(request: Request):
    form_data = await request.form()
    username = str(form_data.get("username", "")).strip()
    password = str(form_data.get("password", "")).strip()

    if not username or not password:
        return templates.TemplateResponse(request, "login.html",
{"request": request, "error": "Please enter both username and
password."})
    if len(password) < 4:
        return templates.TemplateResponse(request, "login.html",
{"request": request, "error": "Password should be at least 4 characters
long."})
    return RedirectResponse(url="/dashboard", status_code=303)
```

login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"
/>
```

```
  <title>Login | EduSphere</title>
  <link rel="stylesheet" href="/static/style.css" />
</head>
<body>
  <div class="auth-shell">
    <div class="auth-card">
      <a class="brand" href="/">EduSphere</a>
      <h1>Welcome back</h1>
      <p>Sign in to continue your learning journey.</p>
      {% if error %}
      <div class="error">{{ error }}</div>
      {% endif %}
      <form method="post" action="/login">
        <label for="username">Email or Username</label>
        <input id="username" name="username" type="text"
placeholder="student@example.com" required />

        <label for="password">Password</label>
        <input id="password" name="password" type="password"
placeholder="Enter password" required />

        <button type="submit">Login</button>
      </form>
      <p class="muted" style="margin-top: 12px;">New here? <a
href="/enroll" style="color: var(--primary); font-weight: 700;">Enroll
now</a></p>
    </div>
  </div>
</body>
</html>
```

8.8 Dashboard and Contact Module Code

```
@app.get("/dashboard", response_class=HTMLResponse)
async def dashboard(request: Request):
    return templates.TemplateResponse(request, "dashboard.html", {
        "request": request,
        "stats": get_stats(),
        "categories": fetch_all("SELECT * FROM categories ORDER BY id"),
        "courses": fetch_all("SELECT * FROM courses ORDER BY id"),
        "enrollments": fetch_all("SELECT * FROM enrollments ORDER BY id
DESC LIMIT 8"),
    })

@app.get("/contact", response_class=HTMLResponse)
async def contact_page(request: Request):
    return templates.TemplateResponse(request, "contact.html",
{"request": request})
```

8.9 JavaScript Code

Displays the current year in the footer of every page that contains an element with the data-year attribute.

```
document.addEventListener("DOMContentLoaded", () => {
  const year = document.querySelector("[data-year]");
  if (year) {
    year.textContent = new Date().getFullYear();
  }
});
```

8.10 Stylesheet Code (Colour Theme)

The colour theme of the application is defined once as CSS variables and reused across all pages.

```
:root {
  --bg: #07111f;
  --surface: rgba(255,255,255,0.96);
  --surface-2: #f7f9ff;
  --primary: #6366f1;
  --primary-dark: #4338ca;
  --accent: #ec4899;
  --accent-2: #06b6d4;
  --text: #0f172a;
  --muted: #64748b;
  --border: rgba(148, 163, 184, 0.24);
  --shadow: 0 20px 45px rgba(2, 8, 23, 0.14);
}
```

9. SAMPLE OUTPUT

9.1 Home Page

Displays the main interface of the Smart Course Enrollment System. The page shows the EduSphere brand, the navigation links, a hero section with the headline “Grow your skills with modern, career-ready courses”, the Start Learning and Explore Courses buttons, and a panel that lists the benefits of the platform.

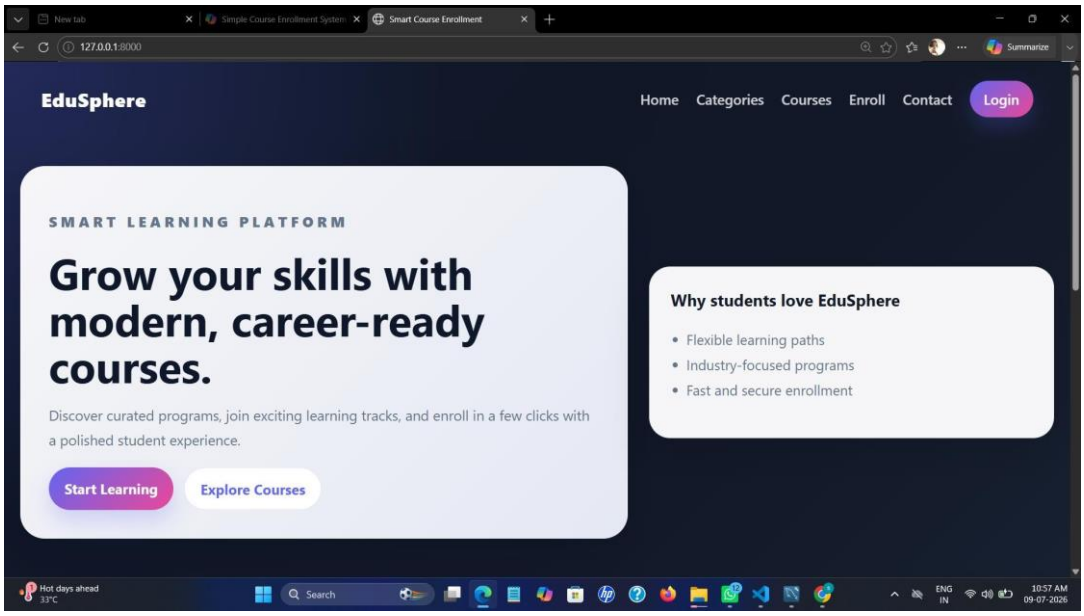


Figure 9.1: Home page

9.2 Login Page

Displays the login form with the fields Email or Username and Password, a Login button, and a link for new users to enroll. Invalid input is shown in an error box above the form.

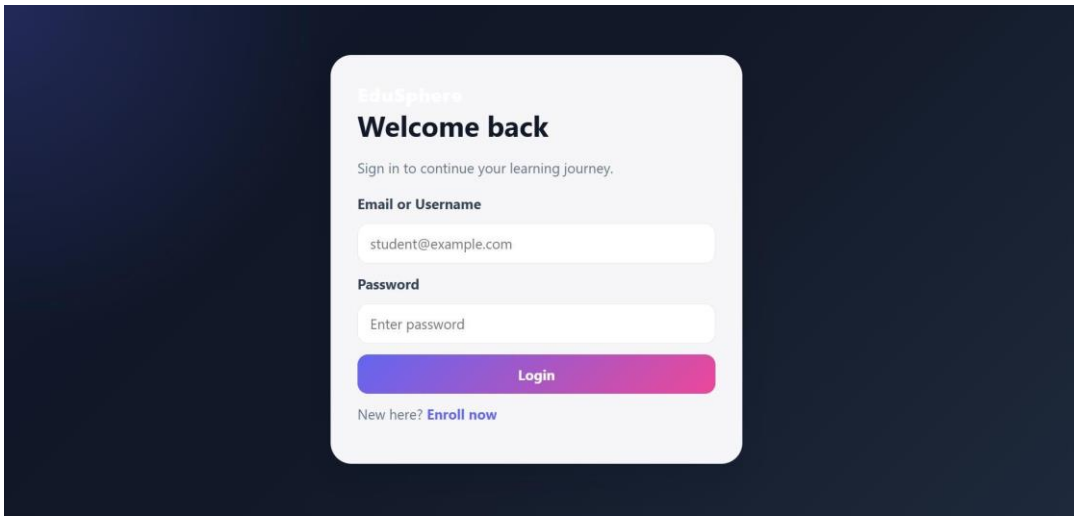


Figure 9.2: Login page

9.3 Categories Page

Displays the five learning categories as cards: Computer Science, Business & Leadership, Design & Creative, Data Science, and Languages & Communication. Each card has an Open category link that leads to the filtered course list.

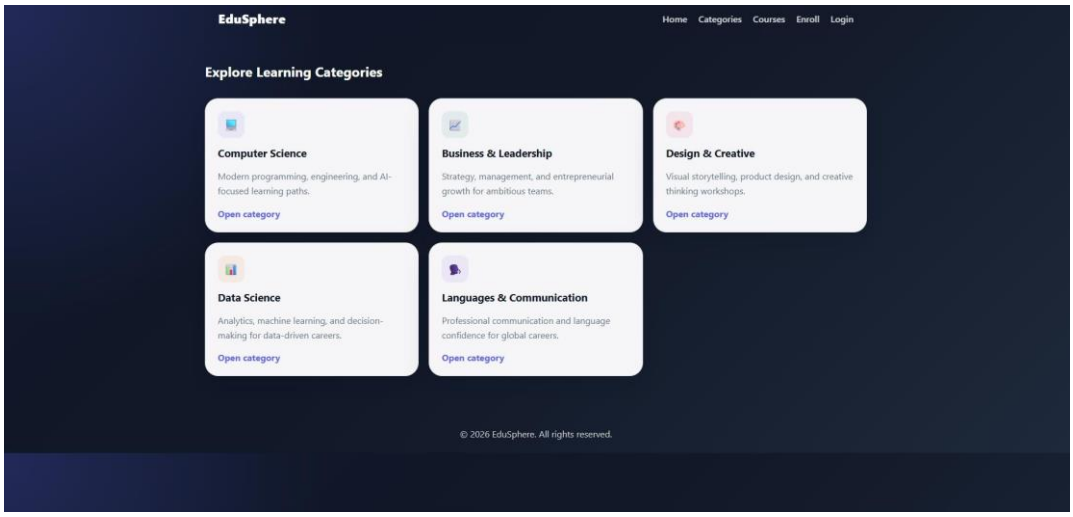


Figure 9.3: Learning categories page

9.4 Course Catalog Page

Displays the course catalog with a category filter. Each course card shows the title, description, category, instructor, duration, level, and number of seats. In this output, the Computer Science category is selected.

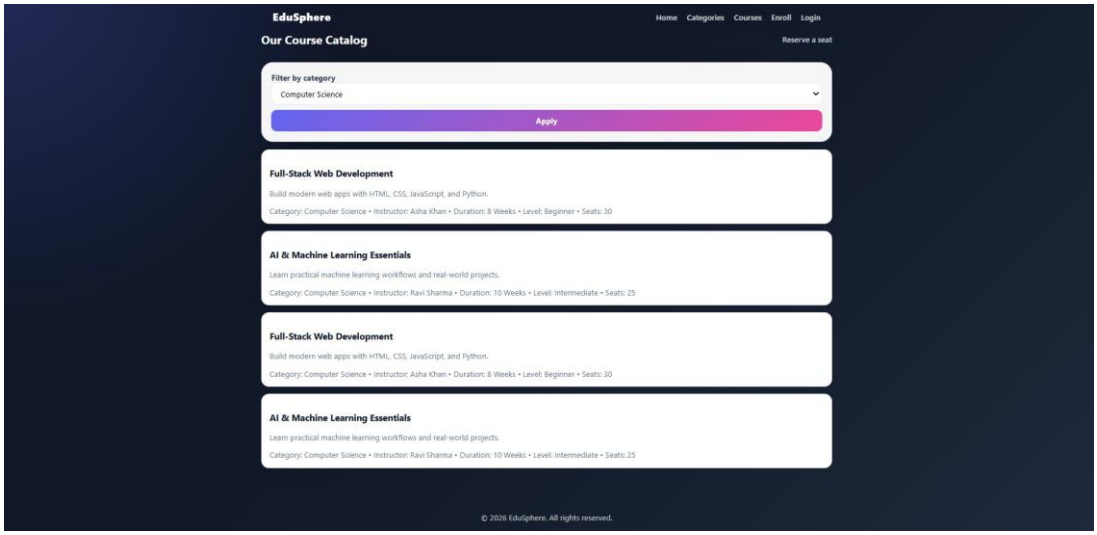


Figure 9.4: Course catalog with category filter

9.5 Enrollment Page

Displays the enrollment form in which learners enter their full name, email address, phone number, category, and preferred course before submitting the request.

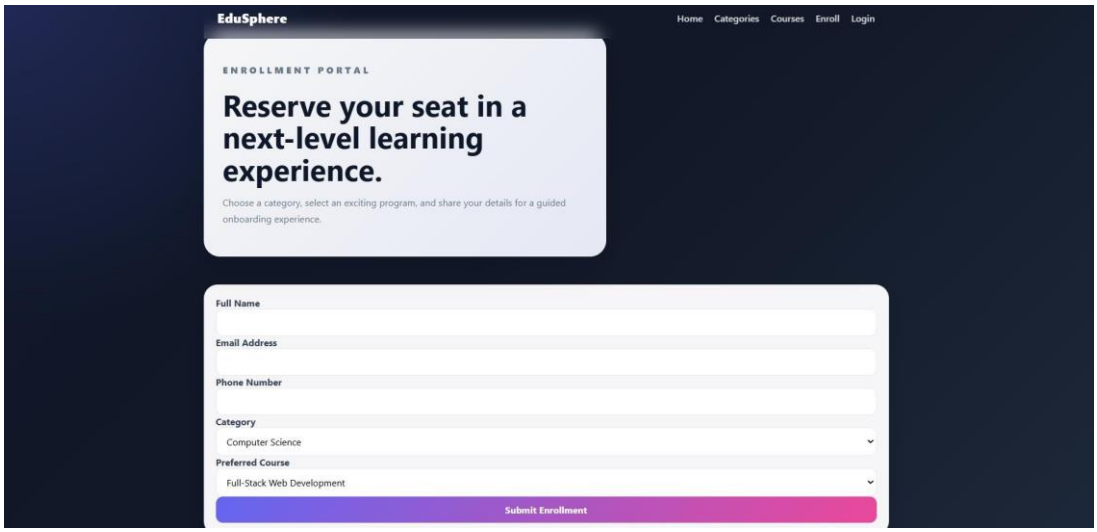


Figure 9.5: Enrollment form

9.6 Successful Enrollment

After a valid submission, the system stores the enrollment in the database and displays the message “Enrollment submitted successfully. We will contact you shortly.”

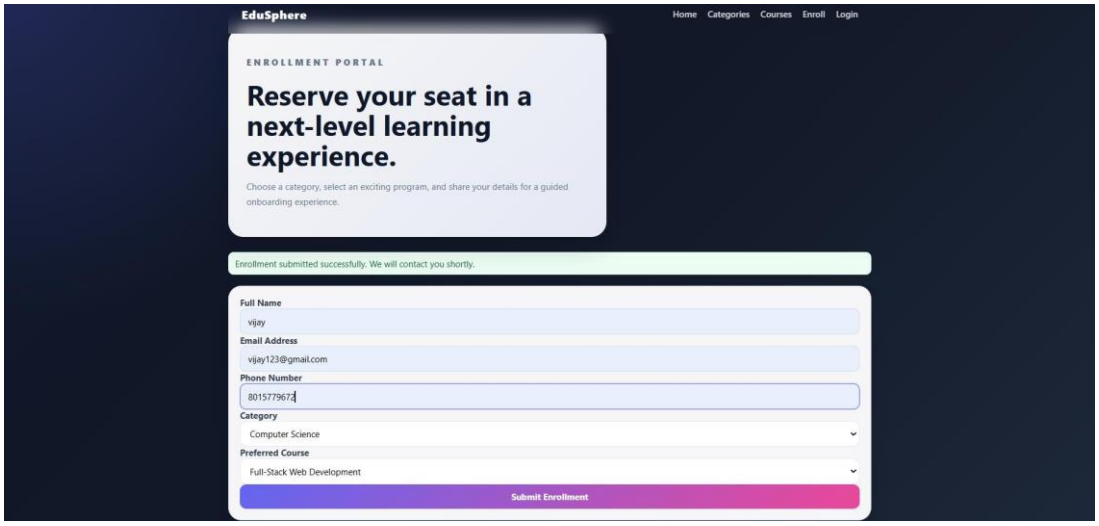


Figure 9.6: Enrollment submitted successfully

10. ANALYSIS

Testing of the Smart Course Enrollment System was carried out module by module to validate both functional correctness and overall system behaviour. Functional analysis confirmed that category browsing, course filtering, enrollment submission, login validation, dashboard display, and database initialisation behaved as expected under normal and edge-case inputs such as empty fields, an invalid email format, a short password, and a database that is not running.

10.1 Observations

Performance observations showed that page loading remained fast for typical operations because the queries are simple and the amount of sample data is small. The responsive, card-based interface adapted well to different screen sizes. The course filter, the enrollment form, and the dashboard reduced the number of steps needed to find a course and register compared to the traditional manual process.

Comparative analysis against the existing manual system highlighted clear improvements. Course information is available in one place, enrollment is reduced to a single form, data entry errors are reduced through form validation, and staff can see the latest enrollments immediately on the dashboard.

Testing also revealed the following points that can be improved in future versions:

- **Duplicate sample courses:** The course list showed each course twice. The most likely reason is that the sample data was loaded twice, once by the SQL script and once by the automatic setup. Adding a unique constraint on the course title and category, or loading the data from only one place, will prevent this.
- **Login validation only:** The login page checks the input but does not verify the credentials against stored accounts.
- **Dashboard count:** The enrollment count is based on the latest eight records, so it cannot exceed eight.
- **Category and course are independent:** The enrollment form allows any course to be selected with any category.
- **Seats are not reduced:** The seat count is displayed but is not reduced after enrollment.
- **Database credentials in code:** The database user and password are written in the source file and should be moved to environment variables.

Overall, the analysis confirms that the system meets its main objectives of reducing manual effort, improving the accuracy of enrollment records, and providing a convenient platform for learners and staff.

11. CONCLUSION

The **Smart Course Enrollment System** successfully demonstrates how full stack web technologies can be applied to modernize and streamline the course enrollment process. By

replacing manual, paper-based registration with a centralized digital platform, the system reduces administrative workload, minimizes human errors, and improves the overall experience of learners.

The project's layered architecture, consisting of a responsive HTML, CSS, and JavaScript frontend and a FastAPI backend with a MySQL database, provides a clear separation of concerns and enables smooth communication through server-rendered pages. Core modules including category browsing, course catalog with filtering, online enrollment, login validation, dashboard, and automatic database initialisation were implemented and tested successfully.

The system provides learners with an easy way to explore categories, compare courses, and submit enrollment requests. At the same time, staff can monitor the latest enrollment requests and the available courses through a single dashboard.

This project also provided valuable practical experience in Python web development, database design, SQL, form handling, template rendering, responsive user interface design, and software testing. The developed system can serve as a foundation for a more advanced and complete enrollment management solution in the future.

12. FUTURE WORK

- Implementation of **secure registration and login** using password hashing and session or JWT-based authentication, with **role-based access control** for learners and administrators.
- Development of an **administrator panel** to add, edit, and delete courses and categories, and to approve or reject enrollment requests.
- Implementation of **seat management** that reduces the available seats after each enrollment, blocks full courses, and supports a waiting list.
- **Duplicate enrollment prevention** using a unique constraint on the student email and course.
- **Dependent dropdown lists** so that the course list changes according to the selected category.
- Use of **foreign keys** in the enrollments table to store the course and student identifiers instead of names.

- Automated **email and SMS notifications** for enrollment confirmation and status updates.
- Integration of a **payment gateway** such as Razorpay or Stripe for paid courses.
- Addition of **search and pagination** for large course catalogs.
- Development of **reports and analytics dashboards** showing enrollments per course and per month, with export to CSV or PDF.
- Moving the database settings to **environment variables** for better security.
- Adding **automated tests** for routes using Pytest and the FastAPI test client.
- **Cloud deployment** using Docker and platforms such as AWS, Azure, or GCP for improved scalability and availability.
- Development of a dedicated **Android and iOS mobile application** for convenient course enrollment.

13. REFERENCES

1. **FastAPI Documentation** – Backend development and web application routing. https://fastapi.tiangolo.com
2. **Uvicorn Documentation** – ASGI server used to run the application. https://www.uvicorn.org
3. **Jinja2 Documentation** – Template engine for rendering HTML pages. https://jinja.palletsprojects.com
4. **Pydantic Documentation** – Data validation and models. https://docs.pydantic.dev
5. **MySQL Documentation** – Relational database management and SQL concepts. https://dev.mysql.com/doc/
6. **MySQL Connector/Python Developer Guide** – Database connectivity from Python. https://dev.mysql.com/doc/connector-python/en/
7. **Python Documentation** – Language features and standard libraries. https://docs.python.org/3/
8. **MDN Web Docs** – HTML, CSS, and JavaScript. https://developer.mozilla.org
9. **W3Schools** – HTML, CSS, and JavaScript learning resources. https://www.w3schools.com

10. **GeeksforGeeks** – Python, SQL, and web development concepts. https://www.geeksforgeeks.org
11. **OWASP Top Ten** – Web application security risks. https://owasp.org/www-project-top-ten/